

Lesson 7.2 — Deployment and Testing

Lesson 7.2 — Deployment and Testing

Code that isn't deployed isn't real, and code that isn't tested isn't trusted. This lesson teaches you to ship the app to the public internet and to write the small set of tests that catch the bugs that actually happen.

By the end of this lesson you will:

- Deploy this Next.js app to **Vercel** through GitHub, with preview deployments on every PR.
- Understand the difference between **build-time** and **run-time** in Next.js.
- Configure **environment variables** safely (you don't have any yet, but you will).
- Set up **Vitest + React Testing Library** for unit tests.
- Write the three tests that actually matter for this app.
- Set up **Playwright** for one end-to-end smoke test.
- Add a **GitHub Actions CI** workflow that runs tests on every push.

This is a long lesson because it covers two big topics. Both are tightly tied together — *you can't deploy with confidence without tests, and there's no point writing tests if you can't ship.*

Part A — Deployment

1. The deployment story for Next.js

Next.js was built by Vercel. They host the framework, they host the docs, they host the deployment platform. **The path of least resistance is Vercel.** Use it for personal projects. Use it for the interview-portfolio version of this app.

For a corporate gig, you might end up on AWS, GCP, Cloudflare, or self-hosted Kubernetes. Those all work — Next.js exports a Node.js server (`next start`) or static files (`next export`). The principles transfer. We're going to use Vercel because it's:

- **Free for hobby projects.**
- **Zero-config** — point it at a GitHub repo and you're done.
- **Preview deployments** — every PR gets its own URL.
- **Built-in CDN** — your global users see fast loads.

Per the project rules, deployment goes through **GitHub** → **Vercel automatically**.

We are *not* using `vercel deploy` from the CLI. We are *not* using Netlify. We push to GitHub; Vercel pulls from GitHub.

2. Connect the repo to Vercel

You only do this once. Walkthrough:

1. Go to vercel.com and sign in with your GitHub account.
2. Click **Add New** → **Project**.
3. Vercel will list your GitHub repos. Pick `reverse-understanding`.
4. Vercel auto-detects Next.js. Leave the build command (`next build`) and output directory (`.next`) at defaults.
5. Click **Deploy**.

Two minutes later you'll have a live URL like `https://reverse-understanding.vercel.app`. Open it. Click around. Confirm the standings render, the analytics charts appear, etc.

If anything is broken on the live site but works locally, **read the build logs first**. Vercel surfaces them in the deployment detail page. The most common causes:

- A `lib/` constant that imports from a Node-only module (e.g., `fs`).
- A component using `'use client'` incorrectly (or *not* using it when it should).
- A TypeScript error that `npm run dev` ignored but `next build` rejected.

3. Preview deployments

Once connected, **every push to a non-main branch** triggers a preview deployment. Vercel comments on the PR with a link.

Try it:

```
git checkout -b polish/error-boundary
# make changes
git add .
git commit -m "Add error boundary"
git push -u origin polish/error-boundary
gh pr create --title "Add error boundary" --body "Wraps the root component."
```

Within ~90 seconds, the PR will have a comment from Vercel: “Deployed to <https://reverse-understanding-git-polish-error-boundary-yourname.vercel.app>”.

You and your reviewers can now click through that URL and verify the change *visually* — without anyone running it locally. This is the workflow that makes design review possible at scale.

4. Environment variables

Right now, this app has no secrets. All data is hardcoded in `lib/`. The day you add a real backend (e.g., fetching tournament data from a Sportradar API), you’ll need to handle:

- An API key.
- An API base URL.
- Maybe a feature-flag value.

In Next.js, env vars come in two flavors:

Prefix	Where it’s available	Example
NEXT_PUBLIC_*	Client <i>and</i> server. Bundled into the JS sent to browsers. Use only for <i>non-secret</i> values.	NEXT_PUBLIC_API_BASE
(no prefix)	Server only. Available in server components, API routes, and <code>getServerSideProps</code> -equivalents.	SPORTRADAR_API_KEY

Rule of thumb: any value the browser must not see goes without the prefix. Anything the browser *does* see — e.g., a public CDN URL — gets the prefix.

4.1. Local `.env.local`

Create `.env.local` in the project root (it’s already gitignored):

```
NEXT_PUBLIC_API_BASE=https://api.example.com
SPORTRADAR_API_KEY=sk_test_abc123
```

Reference these in code:

```
const apiBase = process.env.NEXT_PUBLIC_API_BASE;
const apiKey = process.env.SPORTRADAR_API_KEY; // server only
```

4.2. Vercel environment variables

In the Vercel dashboard: **Project** ▸ **Settings** ▸ **Environment Variables**. Add the same keys, but with the production values. Choose which environments they apply to: Production, Preview, Development.

The day you onboard another developer, they don't need your `.env.local`. They run `vercel env pull` and Vercel fills it in. (We're not using the CLI for deployment. We are allowed to use it for env-var convenience.)

5. Build-time vs. run-time

This trips up most React developers moving to Next.js. Internalize it now.

- **Build time** — when `next build` runs, either locally or on Vercel. The output is a `.next/` directory with prerendered HTML, JS bundles, and a manifest.
- **Run time** — when a user hits the deployed app. Vercel serves the prebuilt assets and runs the Node server for any dynamic pages.

Pages in App Router come in three flavors:

1. **Static** (the default for our app) — rendered at build time, served as static HTML. Fastest possible load.
2. **Dynamic SSR** — rendered on every request from a server component. Slower but always fresh.
3. **Client-only** — `'use client'` at the top means the component runs in the browser only.

Our app is fully static. The `lib/` data is baked into the bundle at build time. To make it *dynamic* — e.g., to fetch the latest match results from an API — we'd convert `app/page.tsx` to be `async` and `await` a `fetch`. That moves it from category 1 to category 2.

Why does this matter for an interview? Because someone *will* ask: “Where does this app get its data?” and “What would you change to support live data?” — and you need to be able to answer with the build-time / run-time vocabulary.

Part B — Testing

6. The testing pyramid (Trophy)

The classical *test pyramid* says: many unit tests, some integration tests, few E2E tests.

The modern *testing trophy* (Kent C. Dodds) says: many integration tests, some unit tests, a small E2E layer, and static checking (TypeScript) underneath everything. We’re using the trophy mental model:

- **Static** — TypeScript already catches a huge category of bugs. **Free testing.** This is why we paid the upfront type-discipline tax in Chapter 1.
- **Unit** — pure functions in `lib/scoring.ts`. Cheap, fast, lots of cases.
- **Integration** — render a component, simulate user clicks, assert what changed. The **highest-value** tier.
- **E2E** — open a real browser, navigate the deployed app, click through. Slow, brittle, but irreplaceable for confidence.

For this app, we’re going to write:

- **3 unit tests** for `scorePick` and `calculateTeamScores`.
- **1 integration test** for the standings tab rendering.
- **1 E2E test** that confirms tab switching works.

That’s five tests total. **It’s enough.** Senior engineers know the value curve drops off fast — your time is better spent writing more *features* than more *tests of features*.

7. Install Vitest + React Testing Library

Vitest is the modern Jest. It’s fast, ESM-native, and integrates with Vite/Next out of the box.

```
npm install -D vitest @vitejs/plugin-react @testing-library/react @testing-library
```

Add a `vitest.config.ts` at the root:

```
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';
import path from 'path';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    setupFiles: ['./vitest.setup.ts'],
  },
  resolve: {
    alias: {
      '@': path.resolve(__dirname, '.'),
    },
  },
});
```

And a vitest.setup.ts:

```
import '@testing-library/jest-dom/vitest';
```

Add to package.json scripts:

```
"test": "vitest run",
"test:watch": "vitest"
```

Run `npm test`. You should see *"No test files found, exiting with code 1."* Good — the framework is wired up.

8. Unit tests for `scorePick` and `calculateTeamScores`

Create `lib/scoring.test.ts`:

```
import { describe, it, expect } from 'vitest';
import { scorePick, calculateTeamScores } from './scoring';
import type { Match, Team } from './types';
```

```

describe('scorePick', () => {
  it('returns 3 when the pick matches the winner', () => {
    const match: Match = { p1: 'A', p2: 'B', winner: 'A', score: '10-5' };
    expect(scorePick('A', match)).toBe(3);
  });

  it('returns 1 when the pick is wrong', () => {
    const match: Match = { p1: 'A', p2: 'B', winner: 'A', score: '10-5' };
    expect(scorePick('B', match)).toBe(1);
  });

  it('returns null when the match has no winner yet', () => {
    const match: Match = { p1: 'A', p2: 'B' };
    expect(scorePick('A', match)).toBeNull();
  });
});

describe('calculateTeamScores', () => {
  it('returns a total of zero for a team with no completed picks', () => {
    const team: Team = {
      name: 'Test Team',
      color: '#000',
      icon: '🏠',
      r1: [],
      r2: [],
      qf: [],
      sf: [],
      final: '',
    };
    const scores = calculateTeamScores(team);
    expect(scores.total).toBe(0);
  });

  it('contains a details array for every round', () => {
    const team: Team = {

```

```

    name: 'Test Team',
    color: '#000',
    icon: '🏆',
    r1: [],
    r2: [],
    qf: [],
    sf: [],
    final: '',
  };
  const scores = calculateTeamScores(team);
  expect(scores).toHaveProperty('r1Details');
  expect(scores).toHaveProperty('r2Details');
  expect(scores).toHaveProperty('qfDetails');
  expect(scores).toHaveProperty('sfDetails');
  expect(scores).toHaveProperty('fDetails');
});
});

```

What's worth noting

- We test the **pure functions** because they're cheap and high-confidence. Pure-function tests are the highest ROI tests in any codebase.
- We test the **shape of the return value**, not specific scores from real data. Tying tests to real-world results is brittle — when the tournament resets next year, your tests break for the wrong reason.
- We test **boundary conditions**: empty arrays, undefined winners. Bugs hide in the boundaries.

Run `npm test`. All three should pass. (Adjust the type imports if your `Team` interface uses slightly different field names.)

9. Integration test for the standings tab

Create `components/tabs/StandingsTab.test.tsx`:


```

import { describe, it, expect } from 'vitest';
import { render, screen } from '@testing-library/react';
import StandingsTab from './StandingsTab';
import { TEAMS } from '@/lib/teams';
import { calculateTeamScores } from '@/lib/scoring';

describe('StandingsTab', () => {
  it('renders every team name', () => {
    const teams = TEAMS.map((t) => ({ ...t, scores: calculateTeamScores(t) })
      .sort((a, b) => b.scores.total - a.scores.total);

    render(<StandingsTab teams={teams} onTeamClick={() => {}} />);

    for (const t of TEAMS) {
      expect(screen.getByText(t.name)).toBeInTheDocument();
    }
  });
});

```

This test does what a real user does:

1. **Renders the component** with realistic props.
2. **Asks the DOM:** “Is the team name on screen?”

If you change the styling, this test still passes. If you accidentally `.filter()` out half the teams, this test fails. **It’s testing behavior, not implementation.**

This is the React Testing Library philosophy in two sentences:

The more your tests resemble the way your software is used, the more confidence they can give you.

— Kent C. Dodds

Memorize that. It’s also a great interview answer to “*How do you decide what to test?*”

10. End-to-end test with Playwright

Install:

```
npm install -D @playwright/test
npx playwright install --with-deps
```

Create e2e/smoke.spec.ts:

```
import { test, expect } from '@playwright/test';

test('user can switch tabs', async ({ page }) => {
  await page.goto('http://localhost:3000');

  await expect(page.getByText('Standings', { exact: false })).toBeVisible();

  await page.getByRole('tab', { name: /matches/i }).click();
  await expect(page.getByText(/Round 1/i)).toBeVisible();

  await page.getByRole('tab', { name: /analytics/i }).click();
  await expect(page.getByText(/points breakdown/i)).toBeVisible();
});
```

Add a playwright.config.ts:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './e2e',
  use: {
    baseURL: 'http://localhost:3000',
  },
  webServer: {
    command: 'npm run dev',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
  },
});
```

Add to package.json:

```
"test:e2e": "playwright test"
```

Run `npm run test:e2e`. Playwright will spin up the dev server, open a real Chromium browser, and walk through the test. Take a moment to watch it; it's satisfying.

What this test catches

- **Hydration errors** — if the server-rendered HTML doesn't match the client tree, Playwright would see a console error.
- **Routing bugs** — if a tab click stops working, the test fails.
- **Missing accessibility roles** — `getByRole('tab')` only finds elements with the right ARIA. Half of writing E2E tests is being forced to add proper a11y, which is a feature.

11. CI with GitHub Actions

Create `.github/workflows/ci.yml`:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm test
```

```
- run: npm run build
```

Push that to GitHub. Open the **Actions** tab. Within ~2 minutes you'll see a green check next to your commit.

Why include `npm run build`?

Because a passing test suite isn't worth much if the production build is broken. The build is itself a giant smoke test — it type-checks, bundles, and lints everything.

Why not run E2E in CI yet?

Playwright in CI requires a Linux runner with browsers preinstalled, plus more orchestration. It's worth doing eventually. For a personal portfolio project, run E2E locally before pushing big changes.

12. The five-test rule

If you take one thing from this lesson:

A small project deserves a small test suite. Don't aim for 100% coverage; aim for the five tests that catch the bugs that actually happen.

For *this* app, those are:

1. `scorePick` returns the right number for the right inputs. (Unit)
2. `calculateTeamScores` doesn't crash on missing data. (Unit)
3. The standings tab renders all teams. (Integration)
4. Tab switching works. (E2E)
5. The build succeeds. (CI)

Anything beyond this is bonus. Ship and iterate.

13. Vibe prompt you would have used

"I have a Next.js 14 app deployed on Vercel via GitHub. Set up Vitest + React Testing Library. Write three unit tests for my pure scoring functions in `lib/scoring.ts` and one integration test that renders `<StandingsTab>` with realistic props and asserts every team name is in the DOM. Then set up Playwright with a single E2E test that opens the home page, clicks the Matches tab, and asserts that 'Round 1' is visible. Finally, write a GitHub Actions workflow that runs lint, test, and build on every push. Use TypeScript everywhere."

Notice:

- **One LLM, four deliverables.** A focused prompt produces focused output.
 - **Specific assertions.** Saying "every team name is in the DOM" gives the LLM a target instead of generating vague test scaffolding.
 - **Toolchain named.** Vitest *and* RTL *and* Playwright *and* Actions. The LLM doesn't have to guess.
-

14. CHECK YOURSELF

- ☐ What's the difference between `NEXT_PUBLIC_*` and an unprefixed env var?
 - ☐ What's a *preview deployment* and why are they valuable?
 - ☐ What's the difference between build-time and run-time in Next.js?
 - ☐ Why is testing pure functions the highest-ROI testing you can do?
 - ☐ What does Kent C. Dodds mean by *"the more your tests resemble the way your software is used, the more confidence they can give you"*?
 - ☐ Why does Playwright's `getByRole( 'tab' )` reward you for writing accessible code?
 - ☐ What's the testing trophy, and how does it differ from the testing pyramid?
 - ☐ Why include `npm run build` in CI?
 - ☐ What are the 5 tests this app needs, and why does it not need more?
 - ☐ How would you switch from build-time data to live data fetched from an API?
-

15. Where you are now

- The app is live on the public internet.

- Every PR gets a preview URL.
- Five tests guard the things that matter.
- CI runs on every push.

You can confidently say *“the app is shipped”* in an interview. Move on to **03-interview-narrative.md** — the closing lesson, where we turn this entire project into a 90-second story you can tell from memory.